Manifesto de Engenharia Agêntica: A Filosofia de Desenvolvimento de Matt Pocock

A ascensão da inteligência artificial generativa não é apenas uma mudança de ferramental; é uma reconfiguração tectônica da engenharia de software. No paradigma atual, a execução tática tornou-se uma commodity de custo zero, forçando o desenvolvedor a abandonar o papel de digitador para assumir o de estrategista. Este manifesto detalha como operar nessa nova realidade, priorizando princípios perenes sobre o hype efêmero dos modelos.

1. O Novo Paradigma: Programação Tática vs. Estratégica

A distinção fundamental, baseada na obra *Philosophy of Software Design* de John Ousterhout, separa o desenvolvimento em dois níveis de consciência:

- **Programação Tática:** Foca na entrega imediata, sintaxe, correção de bugs pontuais e escrita de código. Na era pré-IA, isso consumia 90% do tempo. Hoje, a IA "devorou" essa camada. Ela é mais rápida, barata e eficiente para lidar com a sintaxe do que qualquer humano.
- **Programação Estratégica:** Foca em "vencer a guerra". Envolve o design de sistemas, a definição de interfaces entre módulos, a arquitetura de longo prazo e a velocidade (velocity) do fluxo de trabalho.

E daí? O desenvolvedor que permanece mergulhado na tática — o "code monkey" — tornou-se um passivo. O valor migrou para o **General de Guerra** . Sua função não é mais disparar os tiros (escrever linhas de código), mas gerenciar uma frota infinita de "soldados táticos" (agentes) para garantir que o sistema seja robusto e escalável. O foco humano deve ser, obrigatoriamente, a estratégia.

2. O Conceito de "Harness" (Arreio) vs. O Modelo

Existe uma obsessão cega pelo "Motor" (o modelo de LLM mais recente). No entanto, um Arquiteto de Sistemas sabe que o controle real reside no **Harness** (a infraestrutura de suporte).

- **O Motor:** O LLM é o componente que você não controla. Você está à mercê do progresso das Big Techs.
- **O Harness:** É o ambiente de execução, os prompts, as ferramentas e a estrutura de contexto que você fornece ao modelo.

E daí? Devemos reconhecer a **"Bitter Lesson" (A Lição Amarga)** do aprendizado de máquina: o poder computacional bruto e a escala sempre vencem a otimização manual a longo prazo. No entanto, enquanto esperamos pelo próximo salto de escala, o *harness* é a única variável sob nosso controle. Um suporte bem projetado permite que modelos mais simples e baratos (como o Claude Haiku ou versões anteriores do Opus) superem modelos "inteligentes" operando em bases de código caóticas.

3. O Teto das Habilidades: O Desenvolvedor como Multiplicador

A tese é clara: **suas habilidades técnicas são o teto do que a IA pode realizar** . A IA não cria competência; ela a multiplica.

- **Seniores:** Experimentam um impulso de 10x porque possuem a *sabedoria* para supervisionar a qualidade e o contexto necessário para direcionar a execução.

- **Juniores:** Recebem um ganho marginal, pois a IA não consegue compensar a falta de fundamentos básicos.

A implicação para o mercado é brutal: o domínio do domínio (*subject matter expertise*) e os fundamentos de engenharia de 30 ou 40 anos atrás são mais valiosos agora do que nunca. Sem uma base sólida, você está multiplicando por zero.

4. AX (Agent Experience): Otimizando o Código para Agentes

Se as décadas passadas foram sobre DX (*Developer Experience*), o futuro pertence à **AX (Agent Experience)** . Precisamos construir sistemas que sejam legíveis e operáveis por máquinas.

Princípios de AX Superior:

- **Modularidade e Interfaces Claras:** Contratos bem definidos entre módulos reduzem a necessidade de a IA ler todo o repositório.
- **Test Scenes e Testabilidade:** A IA precisa de ambientes para validar suas próprias mudanças. Se o código é difícil de testar para um humano, é impossível para um agente.
- **Documentação Mínima Suficiente:** Informações que orientem o agente para os pontos de mudança, evitando o desperdício de tokens.

E daí? Uma AX bem executada reduz o "token spend". Quando a IA não precisa "bater a cabeça na parede" tentando entender uma arquitetura acoplada, ela executa tarefas de forma mais barata, rápida e com menos alucinações.

5. Fluxo de Trabalho AFK (Away From Keyboard) e a Gestão de Filas

A engenharia moderna evoluiu de chats interativos para fluxos **AFK (Away From Keyboard)** . O objetivo é a paralelização do "eu".

- **Sand Castle e Execução Segura:** Utilizar sandboxes (Docker/Podman) para que agentes executem código em isolamento, protegendo o ambiente local enquanto trabalham de forma assíncrona.
- **Filas (Queues) vs. Loops:** A ideia de "Agentic Loops" infinitos é frequentemente ineficiente e cara. A abordagem estratégica utiliza **Filas de Tarefas** . Você atua como um Gerente de Filas: prioriza o backlog, define o escopo e revisa os checkpoints.

Ao rodar cinco agentes simultaneamente via GitHub Actions, você deixa de ser um digitador e se torna um orquestrador. A escala não vem de digitar mais rápido, mas de delegar a tática e gerenciar o resultado.

6. A Tríade da Competência: Conhecimento, Habilidade e Sabedoria

Para dominar este novo fluxo, é preciso distinguir as camadas de aprendizado:

1. **Conhecimento:** Entendimento teórico. A IA possui isso em abundância.
2. **Habilidade (Skill):** Memória muscular e prática. A IA está dominando esta camada através de **Procedures** (Procedimentos).
3. **Sabedoria:** O discernimento de *quando* e *como* algo se encaixa no mundo real. Isso exige contexto e experiência prática, algo que a IA não pode emular.

Procedures vs. Abilities: Um erro comum é dar "Abilities" (habilidades autônomas) demais aos agentes. Isso causa o **"Description Leakage"** : o vazamento de descrições de ferramentas para dentro da janela de contexto, poluindo a atenção do modelo e

aumentando custos. O engenheiro sênior prefere **Procedures** — scripts e fluxos que o humano decide quando invocar, mantendo o controle direcional e a pureza do contexto.

7. Conclusão: O Retorno aos Fundamentos

Vencer a guerra do desenvolvimento com IA não significa perseguir cada novo modelo "Fable" ou "Opus" no dia do lançamento. Significa dobrar a aposta no que é perene. Se você tem uma base de código fácil de mudar, você pode usar um modelo "burro" e ainda assim vencer.
A inteligência está na arquitetura, não apenas no chat.

Próximos Passos Práticos: O "Blank Slate"

- **Limpeza Radical (Blank Slate):** Delete seus arquivos .claud.md, agents.md e plugins desnecessários. O excesso de instruções e "habilidades" autônomas está sabotando sua janela de contexto. Comece do zero.
- **Adote Procedures:** Em vez de dar autonomia total à IA, crie procedimentos manuais que você dispara. Mantenha o volante.
- **Delegue a Implementação via AFK:** Configure ferramentas como o Sand Castle ou GitHub Actions para que a implementação tática ocorra enquanto você foca no design do próximo recurso.
- **Foque na Sabedoria:** Pare de estudar sintaxe efêmera. Estude design de sistemas e fundamentos de software. É lá que o seu multiplicador de 10x reside.